Documentation stage de deuxième année

Installation est configuration de Proxmox/Terraform/Ansible

1. Introduction

Cette documentation explique :

1. L'installation de **Proxmox VE**, **Ansible** et **Terraform** sur un serveur Linux.
2. La mise en œuvre d'une automatisation combinée **Terraform + Ansible** pour déployer et configurer automatiquement des machines virtuelles (VM).

PARTIE 1 — Installation de Proxmox, Ansible et Terraform

2. Prérequis

Avant de commencer, assurez-vous d'avoir :

- Un serveur physique compatible virtualisation
- CPU avec **Intel VT-x / AMD-V** activé dans le BIOS
- Minimum **8 Go RAM** (16 Go recommandé)
- Disque SSD recommandé
- Connexion réseau stable
- Une distribution Linux installée (Debian/Ubuntu recommandé pour Ansible et Terraform)

3. Installation de Proxmox VE

Important : Proxmox s'installe généralement **comme OS principal** sur le serveur.

3.1 Télécharger l'ISO

Télécharger Proxmox VE depuis le site officiel.

3.2 Créer une clé USB bootable

Sous Linux :

```
sudo dd if=proxmox-ve.iso of=/dev/sdX bs=4M status=progress
oflag=sync
```

Remplacer /dev/sdX par la clé USB.

3.3 Installation

1. Démarrer sur la clé USB
2. Choisir **Install Proxmox VE**
3. Configurer :
    - disque
    - mot de passe root
    - réseau
    - hostname

3.4 Accès à l'interface web

Une fois installé :

```
https://IP_DU_SERVEUR:8006
```

4. Installation d'Ansible

Sur un serveur Linux Debian / Ubuntu :

```
sudo apt update && sudo apt upgrade -y
sudo apt install ansible -y
```

5. Installation de Terraform

5.1 Ajouter le dépôt HashiCorp

```
sudo apt update
sudo apt install -y gnupg software-properties-common curl
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo apt-key add -
sudo apt-add-repository "deb https://apt.releases.hashicorp.com
$(lsb_release -cs) main"
```

5.2 Installer Terraform

```
sudo apt update
sudo apt install terraform -y
```

PARTIE 2 — Automatisation des VM avec Terraform + Ansible

6. Architecture recommandée

Le workflow recommandé est :

```
Terraform → crée la VM sur Proxmox
Ansible → configure la VM
```

7. Installation du provider Terraform pour Proxmox

Créer un dossier projet :

```
mkdir infra-proxmox
cd infra-proxmox
```

Créer main.tf

```
  GNU nano 8.4                                  main.tf
terraform {
  required_providers {
    proxmox = {
      source  = "telmate/proxmox"
      version = ">= 2.9.0"
    }
  }
}

provider "proxmox" {
  pm_api_url      = var.pm_api_url
  pm_user         = var.pm_user
  pm_password     = var.pm_password
  pm_tls_insecure = true
}

resource "proxmox_lxc" "mon_conteneur_ansible" {
  target_node  = "pve"
  hostname     = "ct-ansible-target"
  ostemplate   = "local:vztmpl/debian-13-standard_13.1-2_amd64.tar.zst"
  password     = "Sio1234*"
  start        = true

  # Configuration R  seau
  vmid = 106
  rootfs {
    storage = "local-lvm"
    size    = "8G"
  }

  network {
    name   = "eth0"
    bridge = "vmbr134"
```

Ou en plus simplifier :

```
terraform {
required_providers {
proxmox = {
source = "Telmate/proxmox"
version = "~> 2.9"
```

```
}
}
}

provider "proxmox" {
pm_api_url = "https://192.168.1.10:8006/api2/json"
pm_user = "root@pam"
pm_password = "motdepasse"
pm_tls_insecure = true
}
```

8. Création d'une VM avec Terraform

Exemple :

```
resource "proxmox_vm_qemu" "vm_ubuntu" {
  name       = "vm-ubuntu-01"
  target_node = "pve"
  clone      = "template-ubuntu"

  cores  = 2
  memory = 4096

  disk {
    size    = "20G"
    type    = "scsi"
    storage = "local-lvm"
  }

  network {
    model  = "virtio"
    bridge = "vmbr0"
  }
}
```

9. Déploiement avec Terraform

Initialisation :

```
terraform init
```

Prévisualisation :

```
terraform plan
```

Déploiement :

```
terraform apply -auto-approve
```

10. Connexion Terraform → Ansible

L'idée est de récupérer l'IP de la VM créée.

Créer outputs.tf :

```
output "vm_ip" {
value = proxmox_vm_qemu.vm_ubuntu.default_ipv4_address
}
```

Afficher :

```
terraform output vm_ip
```

11. Inventory dynamique pour Ansible

Créer inventory.ini

```
[vm]
192.168.1.50 ansible_user=ubuntu
```

12. Playbook Ansible

Créer setup-vm.yml

```
- name: Configuration automatique VM
hosts: vm
become: yes

tasks:
- name: Mise à jour système
apt:
update_cache: yes
upgrade: dist
```

```
- name: Installer Docker
apt:
name: docker.io
state: present

- name: Installer Nginx
apt:
name: nginx
state: present

- name: Démarrer nginx
service:
name: nginx
state: started
enabled: yes
```

13. Exécution Ansible

```
ansible-playbook -i inventory.ini setup-vm.yml
```

14. Automatisation complète Terraform + Ansible

Créer un script deploy.sh

```
#!/bin/bash
terraform apply -auto-approve
IP=$(terraform output -raw vm_ip)

echo "[vm]" > inventory.ini
echo "$IP ansible_user=ubuntu" >> inventory.ini

ansible-playbook -i inventory.ini setup-vm.yml
```

Rendre exécutable :

```
chmod +x deploy.sh
```

Lancer :

```
./deploy.sh
```

15. Workflow final

```
1. Terraform clone la VM template
2. La VM démarre
3. Terraform récupère l'IP
4. Inventory Ansible généré
5. Ansible configure automatiquement
6. VM prête à l'emploi
```

16. Conclusion

La combinaison Proxmox + Terraform + Ansible permet une infrastructure totalement automatisée :

- provisioning
- configuration
- reproductibilité
- scaling rapide

C'est une excellente base pour du DevOps / Infrastructure as Code (IaC).